

[PLEXOS Desktop](#) > [PLEXOS Desktop](#) > [Technical References](#)

 Export as PDF  

Automating PLEXOS

Automate PLEXOS workflows with supported scripting and integration tools.

1. Introduction

PLEXOS is shipped with an automation component that allows you to automate the set up and execution of PLEXOS models and to retrieve the solution data. This article describes how to create a project in Visual Basic for Applications (VBA) or .NET to automate PLEXOS.

2. Linking to PLEXOS

You can automate PLEXOS from either a .NET (e.g. C# or VB.Net) project or COM (e.g. in VB, VBA in Excel):

2.1. Linking from a .NET project

When using a .NET project, you must locate the PLEXOS_NET.Core.dll library, EEUTILITY.dll and EnergyExemplar.PLEXOS.Utility.dll. These libraries are located within the PLEXOS API installation directory and need to be added as references to your project. Additionally, the files 'master.xml' and 'transform.xml' will need to be copied to your project's output directory (the same folder that will contain 'PLEXOS_NET.Core.dll').

If you want to retrieve solution data then you must also add a reference to "adodb" so that database support is available in the project.

2.2. Linking from a .COM project

The PLEXOS Engine component is automatically registered with Windows when PLEXOS installs. All that is required to use the engine from Visual Basic is to add references in the VB project to these libraries:

Microsoft ActiveX Data Objects 2.8 Library

PLEXOS .NET Engine Version X.Y (e.g. Version 9.0)

2.3. Linking from a Python project

To use the PLEXOS API from Python you should install Python for .Net (Homepage: pythonnet.github.io). This can be installed with pip using `pip install pythonnet`. To set up the relevant imports in a Python project, see the following example:

```
import sys
import clr
sys.path += ['C:\\Program Files\\Energy Exemplar\\PLEXOS 10.0 API']
clr.AddReference('PLEXOS_NET.Core')
clr.AddReference('EEUTILITY')
clr.AddReference('EnergyExemplar.PLEXOS.Utility')
import PLEXOS_NET.Core
import EEUTILITY.Enums
import EnergyExemplar.PLEXOS.Utility.Enums as Enums
```

NOTE: Another Python package that can interface to the .Net based API is PyDotnet, but it is not advisable to use this as it is unable to handle parameter lists of more than 9 parameters.

NOTE: You may already have a module installed named clr. This module will conflict with Python for .Net, but it is safe to uninstall this module.

3. Creating a new PLEXOS database

Using the PLEXOS API, you are able to create a new database and populate it using your own scripts.

From a .NET Project

```
' This will be the location of the new database (please ensure the directory exists)
Dim dbFile As String = "C:\Example\database.xml"
' Create a new API instance
Dim api As New PLEXOS_NET.Core.DatabaseCore()
' Additionally, setting the last option to 'true' will overwrite any existing file
api.NewEmptyDatabase(dbFile, False)
' Load the Database file created above
api.Connection(dbFile)
```

From a .COM Project

```
Dim dbFile As String
dbFile = "C:\Example\database.xml"
' Create a new API instance
Dim oDatabase As New PLEXOS_NET.Database
' Additionally, setting the last option to 'true' will overwrite any existing file
api.NewEmptyDatabase (dbFile, False)
' Load the Database file created above
api.Connection (dbFile)
```

4. Modifying a PLEXOS database

With the PLEXOS API, automation scripts can create new objects, define memberships between objects and create properties for objects.

4.1. Adding objects

From a .NET Project

```
' Add a new object
api.AddObject("Gen1", EEUTILITY.Enums.ClassEnum.Generator, True)
api.AddObject("Node1", EEUTILITY.Enums.ClassEnum.Node, True)
```

From a .COM Project

```
api.AddObject("Gen1", PLEXOS_NET.ClassEnum_Generator, True)
api.AddObject("Node1", PLEXOS_NET.ClassEnum_Node, True)
```

From a Python Project

```
# Add a new object  
classDict = api.FetchAllClassIds()  
api.AddObject("Gen1", classDict["Generator"], True)  
api.AddObject("Node1", classDict["Node"], True)
```

4.2. Adding memberships

From a .NET Project

```
' An example of creating a new membership  
api.AddMembership(EEUTILITY.Enums.CollectionEnum.GeneratorNodes,  
"Gen1", "Node1")
```

From a .COM Project

```
' An example of creating a new membership  
api.AddMembership(PLEXOS_NET.CollectionEnum_GeneratorNodes, "Gen1",  
"Node1")
```

From a Python Project

```
# An example of creating a new membership  
collectionDict = api.FetchAllCollectionIds()  
api.AddMembership(collectionDict["GeneratorNodes"], "Gen1", "Node1")
```

4.3. Adding properties

NOTE: Properties can not be updated, only added and removed.

From a .NET Project

```

Dim nBand As Integer = 0
Dim dValue as Double = 1.0
' How to lookup a membership ID, property enum ID and adding a value for that
membership+property
Dim nMemId As Integer =
api.GetMembershipID(EETILITY.Enums.CollectionEnum.SystemGenerators,
"System", "Gen1")
Dim nPropId As Integer = api.PropertyName2EnumId("System", "Generator",
"Generators", "Max Capacity")
If nMemId > 0 AndAlso nPropId > 0 Then
    api.AddProperty(nMemId, nPropId, nBand, dValue, Nothing, Nothing,
Nothing, Nothing, Nothing, Nothing, Nothing,
EETILITY.Enums.PeriodEnum.Interval)
End If

```

From a .COM Project

```

Dim nBand As Integer
Dim dValue as Double
Dim nMemId As Integer
Dim nPropId As Integer
nBand = 0
dValue = 1.0
' How to lookup a membership ID, property enum ID and adding a value for that
membership+property
nMemId =
api.GetMembershipID(PLEXOS_NET.CollectionEnum_SystemGenerators,
"System", "Gen1")
nPropId = api.PropertyName2EnumId("System", "Generator",
"Generators", "Max Capacity")
If nMemId > 0 AndAlso nPropId > 0
Then' Create a new property using the provided membership ID, property ID,
band and value
api.AddProperty(nMemId, nPropId, nBand, dValue, Nothing, Nothing, Nothing,
Nothing, Nothing, Nothing, Nothing, PLEXOS_NET.PeriodEnum.Interval)
End If

```

From a Python Project

```
api.AddProperty(nMemId, nPropId, nBand, dValue, None, None, None, None,  
None, None, None, EEUTILITY.Enums.PeriodEnum.Interval)
```

```
# This will remove all matching property
api.RemoveProperty(nMemId, nPropId, nBand, None, None, None, None, None,
None, None, EEUTILITY.Enums.PeriodEnum.Interval)
# Removing a membership will also remove any properties that belong to it
api.RemoveMembership(collectionDict["GeneratorNodes"], "Gen1", "Node1")
# Removing an object will also remove any memberships and/or properties that
belong to it
api.RemoveObject("Gen1", classDict["Generator"])
```

4.5. Saving the database

```
api.Close()
```

5. Executing a PLEXOS model

To execute a PLEXOS model, either from COM (e.g. VBA) or .NET, you will need to run the PLEXOS Engine's executable process. An example of executing the PLEXOS Engine by command line with arguments:

```
PLEXOS64.EXE <filename> \m <model name> \n
```

An example of the accepted arguments:

```
\m <model name>    Execute the specified Model
\p <project name>   Execute the specified Project
\o <output path>    The selected output path for the solution
\n                Automatically close execution window
\X                Suppress solution writing - used for testing purposes
\?                Displays the help, lists the arguments accepted by the PLEXOS
Engine
NOTE: You may need to include quotes around the filename or model name if
they contain spaces.
```

Executing from .NET

The following example shows how to execute the PLEXOS Engine in .NET, this example is written in VB.NET:

```
Dim strInstallLoc As String = "C:\Program Files\Energy Exemplar\PLEXOS 9.0\"
Dim strEngineName As String = "PLEXOS64.exe"
Dim strModelPath As String = "C:\Temp\3Node\"
Dim strModelFile As String = "3-node.xml"
Dim strModelName As String = "Base"
Dim oEngine As New Process()
' Location of the PLEXOS Engine (ie: "<install location>\PLEXOS64.exe")
oEngine.StartInfo.FileName = System.IO.Path.Combine(strInstallLoc,
strEngineName)
' Folder location of the PLEXOS Model
oEngine.StartInfo.WorkingDirectory = strModelPath
' Arguments used to start the PLEXOS Engine
oEngine.StartInfo.Arguments = String.Format("{{0}} \m {{1}}", strModelFile,
strModelName)
oEngine.Start()
```

Executing from .COM

The Execute method of the Model object invokes the simulation engine on the input database connected to the Model's parent Engine object. The following is an example in Visual Basic using Microsoft Excel.

```
Dim strInstallLoc As String
Dim strModelPath As String
Dim strModelName As String
strInstallLoc = "C:\Program Files\Energy Exemplar\PLEXOS 9.0\PLEXOS64.exe"
strModelPath = "C:\Temp\3Node\3-node.xml"
strModelName = "Base"
Shell strInstallLoc & " " & strModelPath & " \m " & strModelName & " ",
vbNormalFocus
```

Executing from Python

The following example shows how to execute the PLEXOS Engine in Python:


```

import subprocess
#...
strInstallLoc = "C:\\Program Files\\Energy Exemplar\\PLEXOS
9.0\\PLEXOS64.exe"
strModelPath = "C:\\Temp\\3Node\\3-node.xml"
strModelName = "Base"
# Arguments used to start the PLEXOS Engine
subprocess.run([strInstallLoc, strModelPath, "-m", strModelName])

```

6. Executing a PLEXOS model in PLEXOS Connect

In PLEXOS Connect, PLEXOS models are submitted for execution using the PLEXOS APIs rather than using a command line executable.

NOTE: Simulations may not begin for some time after being submitted, depending on the number of simulations already queued and Connect Client availability.

Executing from .NET

This example is written in VB.NET:

```

Dim strSourcePath As String = "C:\\Temp\\3Node\\"
Dim strPlexosFile As String = "3-node.xml"
Dim strModelName As String = "Base"
Dim strArgument As String = String.Format("{{0}} -m {{1}}", strPlexosFile,
strModelName)
Dim oConnect As New PLEXOS_NET.Core.PLEXOSConnect()
oConnect.Connection("Data Source=127.0.0.1:8888;User
Id=user;Password=pass;")
oConnect.AddRun(strSourcePath, New String() {strArgument})

```

Executing from .COM

The following example is written in VBA (Excel):

```

Sub Execute_Click()
    Dim oConnect As New PLEXOS_NET.PLEXOSConnect
    Dim strDataSource As String
    Dim strFileLoc As String
    Dim strArgs() As String
    Dim strRunIndex As String
    On Error GoTo CATCHstrDataSource = "Data Source=127.0.0.1:8888;User
    Id=user;Password=pass;"strFileLoc = "C:\Temp\3Node\oConnect.Connection
    (strDataSource)If oConnect.IsConnectionValid() Then ' Arguments are
    structured like: "<file name>" -m "<model object name>" ReDim
    strArgs(0) strArgs(0) = ""3-node.xml"" -m ""Base"" ' Create a new run for
    simulation (Runs can also be created from a Jobset) strRunIndex =
    oConnect.AddRunByFiles(strFileLoc, strArgs) If strRunIndex = "0" Then '
    Error: The run failed to be added Else ' Success: The run was successfully
    added End IfElse ' Error: Failed to loginEnd IfEND_TRY:Set oConnect =
    NothingExit SubCATCH:Call MsgBox(Err.Description, vbCritical +
    vbOKOnly)Resume END_TRY
End Sub

```

7. Querying solution values

In PLEXOS the method Query() of the Solution class is used to retrieve an ADODB Recordset object containing selected data in the layout of rows and columns specified by arguments to the function. QueryToList() returns a different result structure which is faster to retrieve in .Net and Python applications.

Querying from .NET

An example of a VB.NET function that queries the solution database can be seen below.

```
Dim oSolution As New PLEXOS_NET.Core.Solution()

' Please use the full path to a solution
oSolution.Connection("C:\Temp\3Node\Model Base Solution\Model Base
Solution.zip")

Dim results As SolutionResultList
results =
oSolution.QueryToList(EETILITY.Enums.SimulationPhaseEnum.STSchedule,
    EETILITY.Enums.CollectionEnum.SystemGenerators,
    """,
    """,
    EnergyExemplar.PLEXOS.Utility.Enums.PeriodEnum.Interval,
    EnergyExemplar.PLEXOS.Utility.Enums.SeriesTypeEnum.Properties)
```

Querying from .COM

The following example, using PLEXOS, retrieves a list of Region Prices and writes them to the debug window in a VBA application.

```
Sub GetSolutionData()
    Dim e As New PLEXOS_NET.Solution
    Dim r As ADODB.Recordset
    Dim dProperty As Double
    Dim strModelPath As String
    strModelPath = "C:\Temp\3Node\Model Base Solution\Model Base
Solution.zip"
    ' Make sure the solution gets closed
    On Error GoTo CATCH' Connect to the zipped solution databasee.Connection
(strModelPath)' Determine property enumdProperty =
e.PropertyName2EnumId("System", "Region", "Regions", "Price")' Get the solution
values in a recordsetSet r = e.Query(SimulationPhaseEnum_STSchedule,
CollectionEnum_SystemRegions, "System", "Reg", PeriodEnum_Interval,
SeriesTypeEnum_Properties, dProperty)' Perform any actions on the recordset
before cleaning it upSet r = NothingEND_TRY:' This cleans up any time files
createde.CloseSet e = NothingExit SubCATCH:Call MsgBox(Err.Description,
vbCritical + vbOKOnly)Resume END_TRY
End Sub
```

Querying from Python

In Python ADODB Recordsets should not be used as it is slower than the API's QueryToList(), which returns a different result structure.

```
oSolution = PLEXOS_NET.Core.Solution()
# Please use the full path to a solution
oSolution.Connection("C:\\Temp\\3Node\\Model Base Solution\\Model Base
Solution.zip")
collectionDict = oSolution.FetchAllCollectionIds()
results =
oSolution.QueryToList(EENUMS.Enums.SimulationPhaseEnum.STSchedule,
    collectionDict["SystemGenerators"],
    "",
    "",
    Enums.PeriodEnum.Interval,
    Enums.SeriesTypeEnum.Properties)
```

7.1. Query parameters

SimulationPhase (SimulationPhaseEnum)

This is the enum of the simulation phase data that you are attempting to query. For example, to query the a property in the ST phase you would use SimulationPhaseEnum.STSchedule.

CollectionId (int)

Use FetchAllCollectionIds(). This returns a Dictionary<string, int> where the key is "{ParentClass}{CollectionName}". For example, you may want to retrieve the "Emission" data for a "Region" so you would use the key "RegionEmissions".

ParentName (String)

This is the name of the parent object. For example, using the CollectionId for "RegionEmissions" as the first parameter, you would have to give the name of the "Region" for which you want to query. This might be "South Australia" for example. If 'Nothing' is passed in for the "Parent Name" then all "Parent" objects will be queried. For example, using "RegionEmissions" as the collection id, then all regions will be queried.

ChildName (String)

This is the name of the child object that you want to query. For example, using "RegionEmissions" as the first parameter, the child is "Emission" and so you must put the emission name you wish to query. If a "Child Instance" name is not passed in and "Nothing" is passed in instead, then all emissions in that Region will be queried. You can also allow multiple objects to be queried at once by separating the object names with a "," comma, for example: "Coal, Oil, Gas".

PeriodTypeId(PeriodEnum)

This is the period type which you would like to query the data with. These enumerated values can be found in PeriodEnum.

SeriesTypeId (SeriesTypeEnum)

This controls what columns are in the query that queries the data. These enumerated values can be found in SeriesTypeEnum. For example, if SeriesTypeEnum.Timeslices is used then the data would be arranged with the timeslices as the columns.

PropertyList (String, Optional)

Use FetchAllPropertyEnums(). This returns a Dictionary<string, int> where the key is "{ParentClass}{CollectionName}.{PropertyName}". This is used to query for certain properties. For example, to query the cost of an emission you would use the key "SystemEmissions.Cost".

NOTE: These must be converted to strings, so you will have to do CStr(propertyDict("SystemEmissions.Cost")). You can also allow multiple properties to be queried at once by separating them by a "," comma, for example CStr(propertyDict("SystemEmissions.Price")) & "," & CStr(propertyDict("SystemEmissions.Cost")). If you want to list all properties then pass in "Nothing".

8. Getting a property enum

There are two functions for obtaining a PropertyEnum value.

FetchAllPropertyEnums() returns a Dictionary<string, int> of all PropertyEnum values found in the database, with the key "{ParentClass}{CollectionName}.{PropertyName}"; eg "SystemRegions.Price". The PropertyName2EnumId() function allows the user to pass the parent, child, collection and property as strings and the appropriate enum value will be returned. This value can then be used in the Solution Query functions. For example, the property enum "SystemRegions.Price" can be retrieved by using the following function call:

```
PropertyName2EnumId("System", "Region", "Regions", "Price")
```

This call can be used in conjunction with the Query function. For example you could do something like this:

```
rs = e.Query(  
    collectionDict("SystemRegions"),  
    PeriodEnum.TradingPeriod,  
    SeriesTypeEnum.Values,  
    "WECC",  
    "PG&E",  
    PropertyName2EnumId("System", "Region", "Regions", "Price"))
```

The PropertyName2EnumId function is defined as:

```
PropertyName2EnumId(  
    ByVal strParent As String,  
    ByVal strChild As String,  
    ByVal strCollection As String,  
    ByVal strProperty As String) As Integer
```

9. Getting the collection ID

There are two functions for obtaining a Collection Id value.

FetchAllCollectionIds() returns a Dictionary<string, int> of all Collection Id values found in the database, with the key "{ParentClass}{CollectionName}"; eg "SystemRegions". The CollectionName2Id() function allows the user to pass the {parent class}, {child class}, {collection name} as strings and the appropriate id value will be returned. This value can then be used in the Solution Query functions. The signature for this function is defined as:

```
CollectionName2Id(  
    ByVal strParentClassName As String,  
    ByVal strChildClassName As String,  
    ByVal strCollectionName As String) As Integer
```

The function takes a number of strings, which are required to obtain the correct collection id. An example of how this can be used is:

```
Dim nCollectionEnum as Integer = oSolution.CollectionName2Id("System",  
    "Generator", "Generators")
```

10. Updating a PSSE Power Flow Raw Data file from a PLEXOS solution

The UpdatePSSERawFromSolution function allows users to update a PSSE Power Flow Raw Data file with values from a PLEXOS solution. The solution used must be from a PLEXOS model created from the same PSSE Power Flow Raw Data file as the one being updated, this is done via the PSSE Import feature. The signature for this function is defined as:

```
UpdatePSSERawFromSolution(ByVal strPSSERawFile As String,  
    ByVal strOutputFile As String,  
    ByVal oDateTime As Object,  
    ByVal bLoadData As Boolean,  
    ByVal bGenerationData As Boolean,  
    ByVal bBranchData As Boolean)
```

Where, strPSSERawFile is the full path and filename of the PSSE raw file to be updated, strOutputFile is the filename for the new updated PSSE raw file, ending in ".raw", oDateTime is the period which will be used for populating the raw file, entered as a DateTime object, and the flags bLoadData, bGenerationData, and bBranchData are for enabling which data will be updated from the solution. This function will make an updated copy of the input PSSE Power Flow Raw Data file, writing it to the same directory as the input file. The Raw file's "Load Data" will be updated with the corresponding Node[Load] data, "Generator Data" with the corresponding Generator[Generation] data, and "Branch Data" with any updated Line[Export Limit] and Line[Units Out] data.

11. Including user scripts

If users have any of their own scripts that need to be executed prior to a simulation, or post-simulation, PLEXOS allows for any user defined scripts to run as part of the simulation execution. To include a pre-simulation script as part of the simulation execution simply save the script as Pre-simulation.bat within the same directory as the PLEXOS database being executed. Similarly for post-simulation scripts, just use Post-simulation.bat. If your script is not currently a .bat file you can create a .bat file to execute your script. PLEXOS defines the following environment variables for user scripts to use. DATASET_PATH: the file path and name of the PLEXOS xml database being executed. SOLUTION_0, SOLUTION_1, ..., SOLUTION_N-1: the file paths of the N solutions produced.

An example of how one might use this feature can be found below.

Pre-Simulation.bat

```
@echo *****In Pre-
simulation*****
@echo off
echo DATASET_PATH=[%DATASET_PATH%]

Pushd "C:\DatasetToSQL\"
C:\DatasetToSQL\DatasetToSQL.exe "%DATASET_PATH%" "input-script.sql"
SQLCMD -e -i "C:\DatasetToSQL\input-script.sql" -S .\SQLEXPRESS
@echo
*****
```

Post-Simulation.bat

```
echo *****In Post-
simulation*****
@echo off
@echo DATASET_PATH=[%DATASET_PATH%]
@echo SOLUTION0=[%SOLUTION_0%]

Pushd %SOLUTION_0%
for %i in (*.zip) do set SolutionName=%SOLUTION_0%%~ni%%~xi
@echo %SolutionName%

Pushd "C:\DatasetToSQL\"
C:\DatasetToSQL\DatasetToSQL.exe "%SolutionName%" "solution-script.sql"
SQLCMD -e -i "C:\DatasetToSQL\solution-script.sql" -S .\SQLEXPRESS
@echo
*****
*****
```

In this example Pre-simulation.bat converts the input PLEXOS database into a SQL database, while Post-simulation.bat uses the DataSetToSQL tool to convert the solution file to a SQL database.

12. Creating an MDB solution file

While the MDB output has been deprecated support for MDB is still available. An Access Database (.accdb) can be generated using the DatasetToSQL tool combined with the post simulation feature.

An example of how to create the Post-simulation.bat to enable this feature can be found below.

Post-Simulation.bat

```
@echo *****In Post-
simulation*****
@echo off
REM Please set the following path of the DatasetToSQL tool according to your
environment.
set DatasetToSQL_Path=C:\DatasetToSQLx86\DatasetToSQL.exe
@echo DATASET_PATH=[%DATASET_PATH%]
setlocal enableDelayedExpansion
for /l %%x in (0, 1, 99) do (
if not "!SOLUTION_%%x!"==" (
@echo The solution folder is: !SOLUTION_%%x!
pushd !SOLUTION_%%x!
for %%i in (*Solution.zip) do (
@echo "%DatasetToSQL_Path%" "%i" --target=access
"%DatasetToSQL_Path%" "%i" --target=access
)
popd
)
)
endlocal
@echo
*****
*****
```

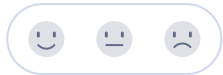
In this example Post-simulation.bat uses the DataSetToSQL tool to create a Microsoft Access (.accdb) database from the solution file.

< Previous
Database Schema

Next >
Customizing PLEXOS

Last updated 1 month ago

Was this helpful?



Additional Resources

[Service Level Agreement \(SLA\)](#)

[PLEXOS Cloud Security and Compliance](#)

© 2026 Energy Exemplar. All rights reserved.